

Name of the Student	T.chaitra
Reg. No	192425217
GitHub Link	https://github.com/chaitramanohar/MLA0402-DEEP-LEARNING

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04

SLOT :

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 1 Hour

Assessment Tool Weightage: 35%

Marks: 50

Code of Conduct:

I T.chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T.chaitra

Name of the student: T.chaitra

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

Signature of the Faculty:

Total Marks :

QUESTIONS:01

Total Marks: 50

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression

Round 1:

Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

Round 2:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	CNN Layer Matcher	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	Filter Finder	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	Parameter Sharing Puzzle	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	CNN Architecture Debugger	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	AlexNet vs ResNet Challenge	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	Edge Detection Explorer	Apply simple (3*3) filters to an image and determine how convolution operations can detect edges and other basic features.
Level 2	Feature Map Detective	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.

Level	Challenge	Description
Level 3	Regularization Rescuer	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	CNN Architecture Race	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	CNN Application Master	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

GAME-BASED LEARNING ACTIVITY: CONVOLUTIONAL NEURAL NETWORKS (CNNs)

Introduction

A **Convolutional Neural Network (CNN)** is a deep learning model mainly used for processing and analyzing images. CNNs automatically learn important visual features such as edges, textures, shapes, patterns, and objects. To understand the fundamental concepts of CNNs, a two-round game-based learning activity is designed. Each round contains five levels, where learners solve challenges related to CNN architecture, filters, parameter sharing, feature extraction, regularization, AlexNet, ResNet, and real-world applications.

The activity follows a progressive learning approach in which learners first understand basic CNN concepts and then apply them to practical problems.

ROUND 1: CNN ARCHITECTURE BUILDER

Level 1 – CNN Layer Matcher

The first level introduces the basic layers of a CNN. The learner must match each layer with its corresponding function.

Important CNN Layers

1. Convolution Layer:

Extracts important features from an image using learnable filters. It detects edges, textures, shapes, and other visual patterns.

2. Pooling Layer:

Reduces the spatial dimensions of feature maps while retaining important information. Max pooling is commonly used.

3. Flatten Layer:

Converts the two-dimensional feature maps into a one-dimensional vector.

4. Fully Connected Layer:

Combines the extracted features and performs classification.

A typical CNN architecture is:

Input → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Level 2 – Filter Finder

A **filter or kernel** is a small matrix used by the convolution layer to detect features in an image.

For example, a 3×3 filter can be used for edge detection:

```
-1 -1 -1
0 0 0
1 1 1
```

The filter moves across the image and performs multiplication and addition with the corresponding pixel values. The resulting values form a **feature map**.

Important convolution parameters include:

- **Filter size:** Determines the size of the kernel.
- **Stride:** Determines how far the filter moves at each step.
- **Padding:** Adds additional pixels around the image to control output dimensions.

The output size can be calculated using:

$$\text{Output Size} = ((N - F + 2P) / S) + 1$$

where:

- N = input size
- F = filter size
- P = padding
- S = stride

Level 3 – Parameter Sharing Puzzle

Parameter sharing means that the same filter weights are reused across different locations of an image.

For example, a 3×3 filter contains:

$3 \times 3 = 9$ weights

If one bias is included:

Total parameters = $9 + 1 = 10$

The same 10 parameters are applied to different parts of the image.

Advantages of Parameter Sharing

1. Reduces the number of trainable parameters.
2. Reduces memory requirements.
3. Reduces computational complexity.
4. Allows the same feature to be detected at different locations.
5. Makes CNNs more efficient than fully connected networks.

For example, an edge-detection filter can detect an edge whether it appears on the left, center, or right side of an image.

Level 4 – CNN Architecture Debugger

In this level, learners identify and correct errors in a CNN architecture.

An incorrect architecture may be:

Input → Pooling → Fully Connected → Convolution → Flatten

A more suitable architecture is:

Input → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

The learner must check:

- Correct layer ordering
- Compatible input and output dimensions
- Appropriate use of pooling
- Correct placement of Flatten
- Suitable output layer

For binary classification, a **sigmoid output** can be used, while **softmax** can be used for multi-class classification.

Level 5 – AlexNet vs ResNet Challenge

The final level of Round 1 compares **AlexNet and ResNet**.

AlexNet

AlexNet is an early and influential deep CNN architecture. It uses convolutional layers, pooling layers and fully connected layers to perform image classification.

ResNet

ResNet, or **Residual Network**, introduces **skip connections** or residual connections. These connections allow information and gradients to pass directly across layers.

The basic residual operation is:

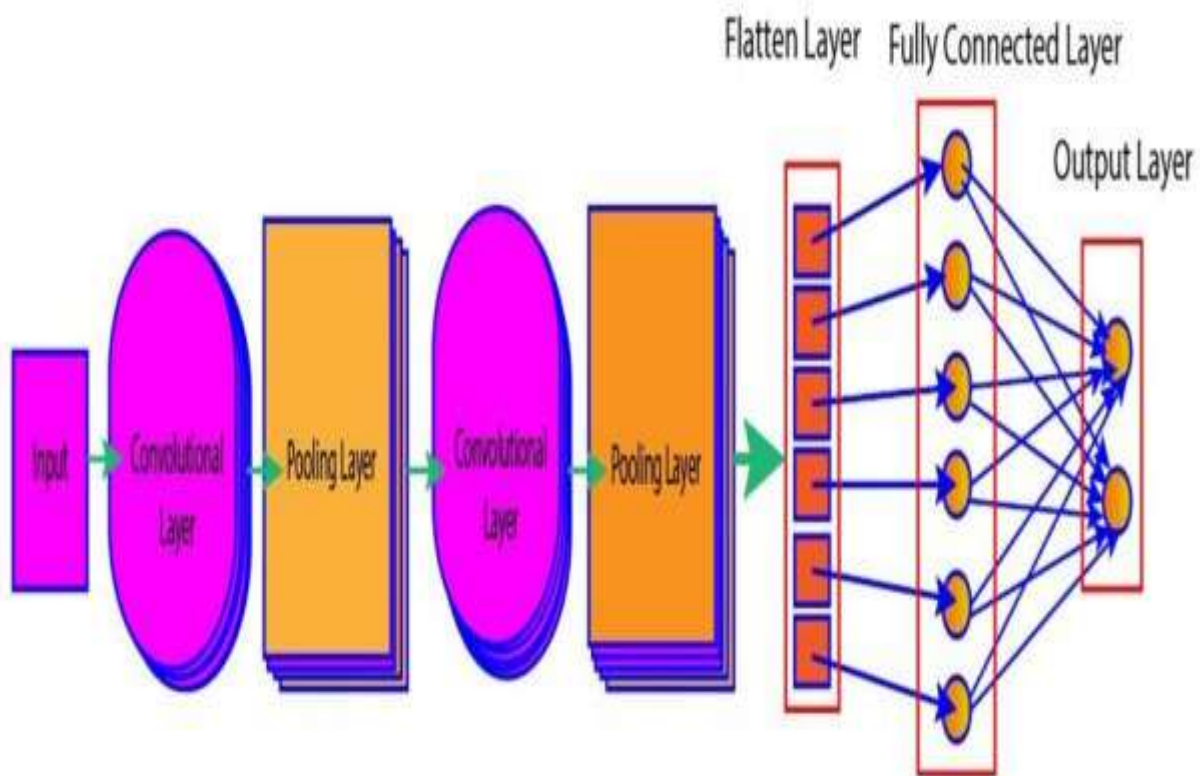
$$\text{Output} = F(x) + x$$

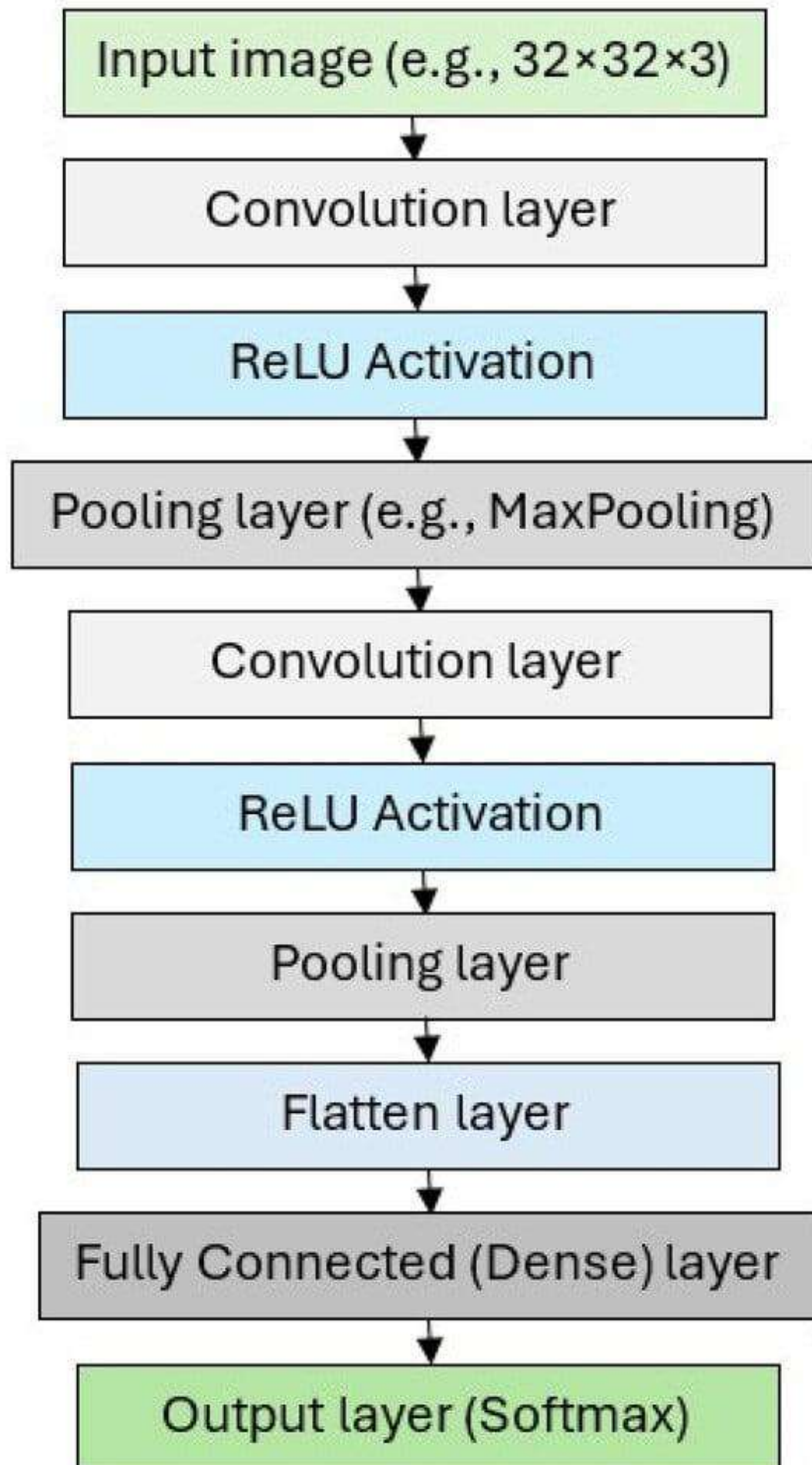
where x is the input and $F(x)$ represents the learned transformation.

Comparison

Feature	AlexNet	ResNet
Architecture	Traditional CNN	Residual CNN
Depth	Relatively shallow	Can be very deep
Skip Connections	No	Yes
Gradient Problem	More susceptible	Reduced
Feature Learning	Good	Very strong
Modern Use	Mostly historical/baseline	Widely used
Accuracy	Good	Generally higher

Therefore, ResNet is generally more suitable for complex image-classification tasks.





ROUND 2: CNN FEATURE DETECTIVE

Level 1 – Edge Detection Explorer

The first level of Round 2 focuses on convolution and edge detection.

A simple 3×3 filter can detect horizontal or vertical edges. When the filter moves across the image, it performs convolution with local regions of pixels.

The basic process is:

Image → Apply Filter → Convolution → Feature Map → Detected Edges

CNNs learn such filters automatically during training rather than requiring all filters to be manually designed.

Level 2 – Feature Map Detective

A **feature map** is the output generated after applying a convolutional filter to an input image.

CNNs learn features progressively through different layers.

Feature progression:

Early Layers → Edges and Lines

Middle Layers → Textures and Shapes

Deep Layers → Object Parts and Complex Patterns

Final Layers → Complete Objects

For example, in an autonomous vehicle:

Edges → Wheels/Windows → Vehicle Parts → Complete Car

Thus, deeper CNN layers learn increasingly complex and meaningful representations.

Level 3 – Regularization Rescuer

Overfitting occurs when a CNN performs very well on training data but performs poorly on unseen test data.

Suitable regularization techniques include:

1. Dropout

Dropout randomly deactivates some neurons during training. This prevents the network from becoming overly dependent on particular neurons.

2. Data Augmentation

New training examples are created by applying transformations such as:

- Rotation
- Scaling
- Cropping
- Translation
- Brightness changes
- Contrast changes

This improves the diversity of the training dataset.

3. Batch Normalization

Batch normalization normalizes activations during training. It improves training stability and can provide some regularization.

4. L2 Regularization

L2 regularization penalizes very large weights and encourages the network to learn simpler representations.

5. Early Stopping

Training is stopped when validation performance stops improving, preventing unnecessary training and overfitting.

Level 4 – CNN Architecture Race

In this level, AlexNet and ResNet are compared for different applications.

For simple image-classification tasks with limited computational requirements, AlexNet may provide a basic solution.

For complex applications requiring strong feature extraction and high classification accuracy, ResNet is generally preferred.

ResNet is particularly useful because its skip connections help solve the **vanishing-gradient problem** and make deeper networks easier to train.

Therefore:

Simple/older baseline → AlexNet

Complex/high-performance application → ResNet

Level 5 – CNN Application Master

The final level requires learners to design a CNN-based solution for a real-world problem.

Example 1: Medical Image Classification

Input: Chest X-ray

Task: Classify normal and disease-affected images

Architecture: ResNet

Output: Normal/Disease

CNN filters can learn important patterns such as edges, textures and abnormal regions.

Example 2: Autonomous Driving

Input: Camera image

Task: Identify cars, pedestrians, bicycles and traffic signs

Architecture: ResNet-based vision system

Output: Object classes and locations

Example 3: Face Recognition

Input: Face image

Task: Extract facial features and identify a person

Architecture: Deep CNN/ResNet-based feature extractor

Output: Identity or feature representation

GAME PROGRESSION SUMMARY

The two-round activity provides progressive learning:

Round 1: CNN Architecture Builder

Basic Layers → Filters → Parameter Sharing → Architecture Debugging → AlexNet vs ResNet

↓

Round 2: CNN Feature Detective

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

↓

CNN MASTER

By completing all five levels in both rounds, learners understand how CNNs are constructed, how convolutional filters extract features, how parameter sharing reduces complexity, how regularization prevents overfitting, how AlexNet differs from ResNet,

and how CNNs are applied to real-world problems such as medical imaging and autonomous vehicles.



Conclusion

The game-based learning activity provides an interactive way to understand CNN concepts. Instead of learning CNN theory only through lectures, learners actively solve problems related to architecture, filters, feature maps, parameter sharing and regularization. The two-round structure gradually progresses from fundamental concepts to practical applications. By the end of the activity, learners can design suitable CNN architectures and select appropriate models such as AlexNet or ResNet for different real-world applications.